
VS Code

VS Sir

AUTOMATION • N8N • AI AGENTS • GITHUB • PYTHON

AUTOMATION MASTERY

Learn Automation from Zero

Python • APIs • Git • GitHub • GitHub Actions

A practical, mobile-first guide for beginners, creators,
educators & professionals

Combined Digital Edition

Volumes 1-3

Vijay Kumar Sharma (VS Sir)

ABOUT THIS EDITION

Automation Mastery combines the first three volumes into one continuous learning journey. It takes you from automation thinking and computer foundations through Python, JSON, HTTP, APIs, Git, GitHub and GitHub Actions.

WHAT YOU WILL LEARN

- How to identify tasks worth automating
- The seven-stage automation pattern: Input → Trigger → Processing → Decision → Action → Output → Monitoring
- Files, folders, paths, terminal and VS Code
- Python patterns used in real automation
- JSON, HTTP requests, APIs and error handling
- Git, GitHub, branches, commits, pull requests and releases
- GitHub Actions workflows, runners, secrets and scheduled automation

HOW TO USE THE BOOK

Learn → Practice → Build. Do the Practical Exercise after each chapter before moving to the Challenge. Keep one folder such as `automation_practice` and build everything inside it. The aim is not to memorise commands; it is to understand patterns well enough to build and troubleshoot your own workflows.

IMPORTANT

Software interfaces, action versions and cloud services change over time. Treat code examples as learning templates and verify current tool documentation when you implement them in a live project. Never place real passwords, API keys or tokens directly in code or public repositories.

© 2026 Vijay Kumar Sharma (VS Sir). Digital educational edition.

TABLE OF CONTENTS

PART 1 — AUTOMATION FUNDAMENTALS

Chapter 1 — What Is Automation?

Chapter 2 — Automation Thinking

PART 2 — COMPUTER & PROGRAMMING FOUNDATIONS

Chapter 3 — Essential Computer Concepts

Chapter 4 — Terminal / Command Line

Chapter 5 — VS Code

PART 3 — PROGRAMMING FOR AUTOMATION

Chapter 6 — Python Fundamentals

Chapter 7 — Python for Automation

PART 4 — DATA FORMAT & INTERNET FUNDAMENTALS

Chapter 8 — JSON

Chapter 9 — HTTP & Internet

Chapter 10 — APIs

PART 5 — GITHUB

Chapter 11 — Git Fundamentals

Chapter 12 — GitHub

Chapter 13 — GitHub Actions

THE LEARNING PATH

Automation thinking → Computer foundations → Python → JSON/HTTP/APIs → Git/GitHub → GitHub Actions → Next: n8n, business tools, databases, AI automation, AI agents, RAG, Docker, cloud, security and real-world projects.

READER FEEDBACK & SUPPORT

Your feedback will help improve the next edition. Please report spelling mistakes, unclear explanations, broken code examples, outdated screenshots, or topics you want added.

WHATSAPP FOR REVIEW

+91 96024 05311

Message format: Automation Mastery Review — Chapter / Page / Issue

Please do not send passwords, API keys, payment credentials or other confidential information.

YOUR 30-DAY PRACTICE TARGET

Week 1: Map repetitive tasks using the seven-stage automation pattern.

Week 2: Become comfortable with terminal, VS Code and Python basics.

Week 3: Build a small file/CSV/API automation and add logging and error handling.

Week 4: Put the project on GitHub and run one GitHub Actions workflow.

BUILD SOMETHING REAL.

The goal of this book is not to finish pages. The goal is to finish working automations.

PART 1

AUTOMATION FUNDAMENTALS

What Is Automation? → Automation Thinking
Building the mindset before touching any tool.

VS Code | VS Sir

CHAPTER 1

What Is Automation?

Automation

Simple meaning: Getting a computer to do a task for you, automatically, without you repeating the same manual steps every time.

Why it matters: It saves time, reduces mistakes, and lets you focus on work that actually needs a human.

Example: Instead of manually copying 50 form responses into a spreadsheet every day, a script or workflow does it the instant a new response arrives.

MANUAL WORK vs AUTOMATED WORK

Manual work requires a human to notice something needs doing, and to do every step by hand, every single time.

Automated work requires a human to design the process ONCE; after that, the computer repeats it forever, correctly, without being asked.

TYPES OF AUTOMATION

Rule-based automation

Simple meaning: Automation that follows fixed IF/THEN rules you define.

Why it matters: It's predictable and easy to debug — the best starting point for most beginners.

Example: "IF a new email arrives with subject 'Invoice', THEN save the attachment to a folder."

Event-based automation

Simple meaning: Automation that runs the moment something happens (an 'event'), rather than on a timer.

Why it matters: It reacts instantly, instead of making you wait for the next scheduled check.

Example: A workflow that runs the second a customer submits a form.

Scheduled automation

Simple meaning: Automation that runs at fixed times, regardless of what else is happening.

Why it matters: Perfect for regular reports or cleanup tasks that don't need to react instantly.

Example: A script that emails a sales report every day at 6 PM.

Data-driven automation

Simple meaning: Automation whose behaviour changes based on the actual data it receives.

Why it matters: Real-world tasks rarely have just one outcome — your automation needs to branch.

Example: "IF the invoice amount > \$1000, route it to a manager for approval."

AI-powered automation

Simple meaning: Automation that uses an AI model to understand, summarise, decide, or generate content as one step in the process.

Why it matters: Some tasks (reading messy text, deciding tone, summarising) are too flexible for simple IF/THEN rules alone.

Example: An AI reads an incoming support email and drafts a suggested reply.

Human-in-the-loop automation

Simple meaning: Automation that pauses at a key moment and waits for a human to approve or edit before continuing.

Why it matters: It combines automation's speed with human judgment on the decisions that matter most.

Example: A workflow drafts a refund but a human must click 'Approve' before money actually moves.

EVERYDAY AUTOMATION EXAMPLES

- **Email automation** — auto-replies, auto-sorting, auto-forwarding.
- **Student attendance** — a scanned QR code instantly updates a spreadsheet.
- **Lead collection** — a website form automatically creates a CRM entry.
- **Google Sheets updates** — new data appended without manual typing.
- **Report generation** — a weekly PDF report built and emailed automatically.
- **Social media workflow** — one post scheduled across multiple platforms.
- **File organization** — downloads automatically sorted into folders by type.
- **Invoice processing** — a PDF invoice automatically logged and categorised.
- **Teaching material generation** — worksheets auto-created from a question bank.
- **Student enquiry management** — new enquiries auto-routed to the right staff member.

■ QUICK TIP

Notice the pattern in every example above: something happens (a trigger), then a fixed sequence of steps runs automatically. That pattern is the entire subject of this book.

■ REMEMBER

Automation does not replace thinking ? it replaces repeating. You design the process once; the computer executes it automatically, while monitoring and handling errors.

EXERCISES

Quick Quiz

- What is the main difference between manual and automated work?
- Give one example each of event-based and scheduled automation.
- What does 'human-in-the-loop' mean?
- Why might a business prefer rule-based automation over AI-powered automation for a simple task?
- Name three everyday tasks in your own life that could be automated.

Practical Exercise

Write down, in plain English, three repetitive tasks you personally do every week, and classify each as rule-based, scheduled, or event-based.

Challenge

Pick your most time-consuming weekly task and sketch (on paper) the trigger and the exact steps a computer would need to follow to do it for you.

Interview Questions

- How would you explain automation to someone with zero technical background?
- What's the difference between automation and AI?
- Describe a real automation you've seen or used.

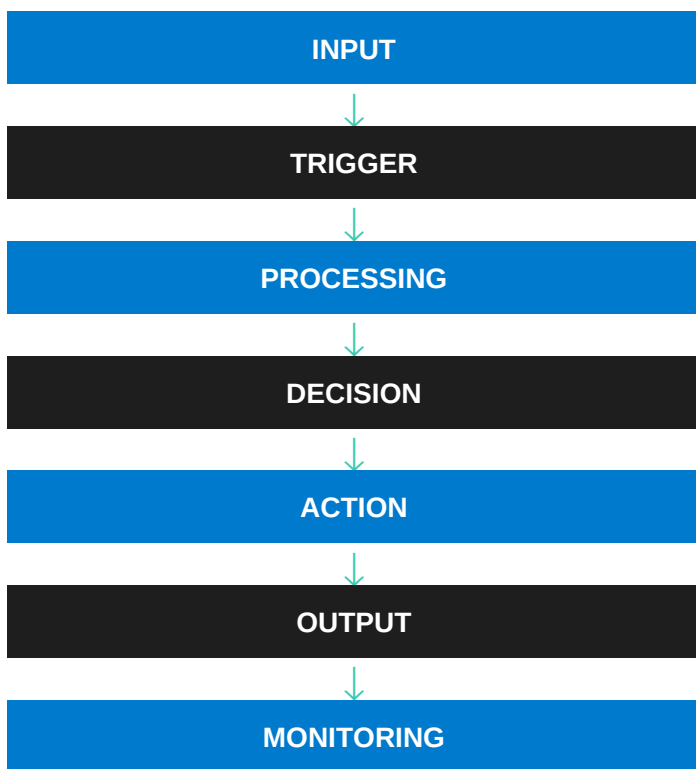
NEXT STEP: Chapter 2 teaches you how to spot automation opportunities and judge whether they're actually worth building.

CHAPTER 2

Automation Thinking

THE UNIVERSAL AUTOMATION DIAGRAM

Every automation you will ever build — no matter how simple or advanced — fits this same seven-stage shape:



- **Input** — the raw information available (a form, a file, an email).
- **Trigger** — the exact moment the automation should start.

- **Processing** — cleaning, transforming, or combining the input data.
- **Decision** — the branching logic (IF/THEN) applied to that data.
- **Action** — the real-world effect (send, save, notify, update).
- **Output** — the final result the automation produces.
- **Monitoring** — knowing whether it actually worked, and being alerted if not.

■ REMEMBER

Before opening any tool, describe your automation using these seven words. If you can't fill in all seven, the idea isn't ready to build yet.

WHAT SHOULD BE AUTOMATED?

- Tasks you repeat often (daily/weekly).
- Tasks with clear, consistent rules.
- Tasks that are boring but necessary.
- Tasks prone to human error when done manually.
- Tasks that must happen fast, faster than a human can react.

WHAT SHOULD NOT BE AUTOMATED?

- Tasks requiring deep human judgment or empathy (e.g. a sensitive parent conversation).
- Tasks that happen only once and never repeat — automating them wastes more time than it saves.
- Tasks where the rules are unclear or constantly changing.

- Tasks with serious consequences if automation gets it wrong, without a human check.

■ WARNING

A common beginner mistake is automating a task that happens only once or twice a year — the time spent building the automation outweighs the time it ever saves.

PAYBACK PERIOD

■ CONCEPT

Payback period estimates how long the time saved by an automation takes to recover the time spent building it.

```
Time saved per run    = manual_minutes - automated  
_minutes
```

```
Total time saved      = time_saved_per_run x runs_  
per_month
```

```
Building time         = hours needed to design + b  
uild + test
```

```
Payback (months) = building_time_hours*60 / total  
_time_saved_per_month
```

If a task takes 10 minutes manually, 1 minute once automated, and happens 20 times a month, that's 180 minutes (3 hours) saved every month. If building it takes 4 hours, it pays for itself in under two months.

AUTOMATION OPPORTUNITY CHECKLIST

- ■ Is this task repeated regularly?
- ■ Are the steps consistent every time?
- ■ Is the trigger clear and detectable by a computer?
- ■ Would time saved justify the building effort (use the payback calculation)?
- ■ Are the risks of a mistake acceptable, or can a human-in-the-loop step reduce that risk?
- ■ Do the tools/data needed already exist, or are they realistic to obtain?

OTHER BENEFITS BEYOND TIME

- **Error reduction** — computers don't get tired or distracted mid-task.
- **Cost reduction** — less staff time spent on repetitive work.
- **Reliability** — the same steps run correctly every single time.
- **Scalability** — handling 10 requests or 10,000 requires no extra manual effort.

■ PRACTICE

Take one task from your Chapter 1 exercise, run it through the **payback calculation** with realistic numbers, and decide honestly whether it's worth automating.

EXERCISES

Quick Quiz

- List the seven stages of the Universal Automation Diagram in order.
- Give one example of a task that should NOT be automated, and explain why.
- What two numbers do you need to calculate time saved per run?
- Name two benefits of automation besides saving time.
- Why is 'monitoring' an essential stage, not an optional extra?

Practical Exercise

Map one real task from your own life or work onto the seven-stage diagram: Input, Trigger, Processing, Decision, Action, Output, Monitoring.

Challenge

Using the `payback calculation` compare two different automation ideas you have and decide which one to build first, based on payback time.

Interview Questions

- How do you decide whether a task is worth automating?
- What's a real example of a task that should NOT be fully automated?
- Explain the automation `payback calculation` in your own words.

NEXT STEP: Part 2 builds the computer foundations — files, terminal, and VS Code — needed before writing any automation.

PART 2

Computer Concepts → Terminal → VS Code
The workspace every automation builder needs to know.

VS Code | VS Sir

CHAPTER 3

Essential Computer Concepts

File

Simple meaning: A single unit of stored data — a document, image, script, or spreadsheet.

Why it matters: Almost every automation reads from or writes to files.

Example: invoice.pdf, report.csv, script.py

Folder (Directory)

Simple meaning: A container that organises files and other folders.

Why it matters: Automations need to know exactly where to find or save files.

Example: Documents/Invoices/2026/

Path

Simple meaning: The exact address of a file or folder on a computer or server.

Why it matters: Automation scripts must reference the correct path, or they'll fail to find the file.

Example: C:\Users\Sir\Documents\report.csv (Windows) or /home/sir/report.csv (Mac/Linux)

Extension

Simple meaning: The letters after the dot in a filename, showing its type.

Why it matters: Different tools expect different extensions — a script needs .py, not .txt.

Example: report.csv, script.py, workflow.json

Environment variable

Simple meaning: A named value stored outside your code, available to programs running on that machine.

Why it matters: It's the standard, secure way to store settings and secrets without hard-coding them into your script.

Example: API_KEY=YOUR_API_KEY

Process

Simple meaning: A running instance of a program.

Why it matters: Automation often means starting, stopping, or monitoring processes.

Example: Your Python script running is one process; your browser is another.

Service

Simple meaning: A program designed to run continuously in the background.

Why it matters: Automation tools like n8n typically run as a service, always ready to react.

Example: A web server or database quietly running 24/7.

Server

Simple meaning: A computer (often remote) that provides something to other computers — data, a website, or a running application.

Why it matters: Automation tools are frequently hosted on a server so they run even when your own laptop is off.

Example: The computer that hosts your n8n workflows in the cloud.

Local machine

Simple meaning: Your own computer, sitting in front of you.

Why it matters: Great for learning and testing, but it must stay on for scheduled automations to run.

Example: The laptop you're using right now.

Cloud

Simple meaning: Computing resources (storage, servers) rented over the internet instead of owned physically.

Why it matters: Cloud-hosted automation keeps running even when your own device is off.

Example: n8n Cloud, GitHub, Google Drive.

Browser

Simple meaning: The application used to view websites and web apps.

Why it matters: Many automation platforms (including n8n) are configured through a browser interface.

Example: Chrome, Firefox, Edge, Safari.

Terminal / Command line

Simple meaning: A text-based way to control your computer by typing commands instead of clicking.

Why it matters: It's faster, scriptable, and required for many automation and developer tools.

Example: Covered fully in Chapter 4.

Shell

Simple meaning: The program that reads and runs the commands you type into a terminal.

Why it matters: Different operating systems use different shells with slightly different commands.

Example: Bash (Mac/Linux), PowerShell/CMD (Windows).

WINDOWS vs macOS vs LINUX

- **Windows** — uses backslashes in paths (C:\Users\...) and PowerShell/CMD by default.

macOS ? built on Unix; uses forward slashes (/Users/...). Zsh is the default shell on modern macOS; Bash is also available.

- **Linux** — the standard OS for servers; also Unix-based, forward slashes, Bash shell — most cloud automation ultimately runs here.

■ QUICK TIP

Most professional automation (n8n servers, GitHub Actions runners, Docker containers) runs on Linux, even if you develop on Windows or macOS — that's why Chapter 4 focuses on commands that work everywhere.

■ REMEMBER

Files, folders, and paths are the 'nouns' of automation — almost every workflow eventually reads from or writes to one.

EXERCISES

Quick Quiz

- What is the difference between a file and a folder?
- Why shouldn't secrets be hard-coded directly into a script?
- What's the difference between your local machine and the cloud?

- Which shell is standard on Linux and macOS?
- Name one reason automation is often hosted on a server rather than a personal laptop.

Practical Exercise

On your own computer, locate the full path of any one file (right-click → Properties on Windows, or right-click → Get Info on Mac) and write it down exactly.

Challenge

Create a folder structure on your computer that could organise a teaching automation project: e.g. project/data, project/scripts, project/output.

Interview Questions

- Explain the difference between a process and a service.
- Why do most automation platforms ultimately run on Linux servers?
- What is an environment variable used for?

NEXT STEP: Chapter 4 gets hands-on with the terminal — your first real automation tool.

CHAPTER 4

Terminal / Command Line

Terminal

Simple meaning: A window where you type text commands instead of clicking icons.

Why it matters: It's faster for repetitive tasks and required for installing and running most automation and developer tools.

Example: Opening 'Terminal' on Mac, or 'Windows Terminal' on Windows.

CMD

Simple meaning: The classic Windows command-line program.

Why it matters: It's the most basic Windows shell, still used for simple commands.

Example: Search 'cmd' in the Windows start menu.

PowerShell

Simple meaning: A more powerful, modern Windows shell.

Why it matters: It's the recommended shell for Windows automation and scripting today.

Example: Search 'PowerShell' in the Windows start menu.

Bash

Simple meaning: The standard shell on Linux and (traditionally) macOS.

Why it matters: Almost every server, Docker container, and GitHub Actions runner uses Bash — learning it transfers

everywhere.

Example: The default terminal on most Linux distributions.

ESSENTIAL COMMANDS

These commands work (with minor naming differences) across Bash, PowerShell and CMD. Practice each one slowly.

pwd (Mac/Linux) / cd (Windows, no arguments)

```
pwd
```

What it does: prints your current folder location. **Expected result:** a full path is printed, e.g. /home/sir/projects.

cd (change directory)

```
cd projects
```

What it does: moves you into the 'projects' folder. **Expected result:** no output, but pwd now shows the new location. **If it fails:** 'No such file or directory' means the folder name or location is wrong — check spelling and capitalisation.

ls (Mac/Linux) / dir (Windows)

```
ls
```

What it does: lists everything inside the current folder. **Expected result:** a list of file and folder names.

mkdir (make directory)

```
mkdir automation_project
```

What it does: creates a new folder with that name.

touch (Mac/Linux)

```
touch script.py
```

What it does: creates a new, empty file. On Windows PowerShell, use 'ni script.py' instead.

cp (copy) / mv (move or rename) / rm (remove)

```
cp report.csv backup_report.csv      # copy
mv report.csv archive/report.csv      # move/rename
rm old_file.txt                       # delete
```

■ WARNING

rm deletes files permanently in most terminals — there is no 'Recycle Bin' safety net. Double-check the filename before pressing Enter.

echo, cat, clear

```
echo "Hello Automation"              # prints text to the screen
cat report.csv                       # displays a file's contents
clear                                 # clears the terminal screen
```

python, pip, git

```
python script.py                    # runs a Python script
pip install requests                # installs a Python package
git status                          # checks a Git repository's status
```

These three commands bridge into the next chapters: Python (Part 3), pip (package installation), and Git (Part 5).

■ DEBUG

If 'python' gives 'command not found', Python may not be installed or not added to your system PATH — reinstall it and make sure the 'Add to PATH' option is checked during setup.

■ REMEMBER

Terminal commands are unforgiving about exact spelling and spacing — but that precision is exactly what makes them scriptable and automatable.

EXERCISES

Quick Quiz

- What command shows your current folder location?
- What is the Windows equivalent of the Mac/Linux 'ls' command?
- What does 'mkdir' do?
- Why is 'rm' considered dangerous?
- What does 'pip install requests' do?

Practical Exercise

Open your terminal, create a folder called 'automation_practice', move into it, and create an empty file called notes.txt inside it.

Challenge

Using only terminal commands, create three subfolders (data, scripts, output) inside automation_practice, then list the contents to confirm.

Interview Questions

- Why do developers prefer the terminal over clicking through folders for repetitive tasks?
- What's the difference between Bash and PowerShell?
- How would you safely test a command that deletes files?

NEXT STEP: Chapter 5 sets up VS Code — the editor you'll use for every script and workflow file in this book.

CHAPTER 5

VS Code

VS Code

Simple meaning: A free, extremely popular code editor made by Microsoft.

Why it matters: It gives you a terminal, file browser, Git tools, and code editing all in one place — exactly what an automation builder needs.

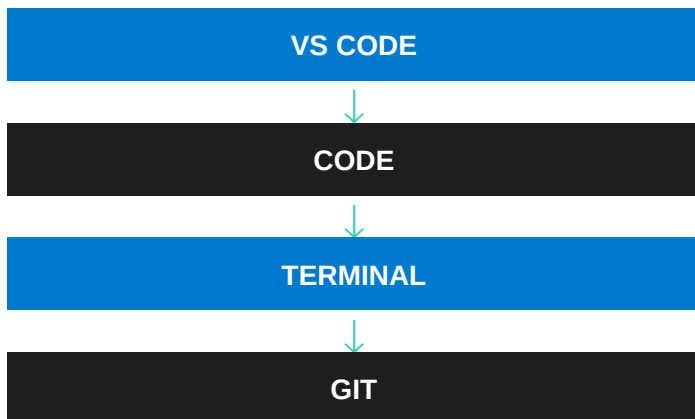
Example: Download from code.visualstudio.com.

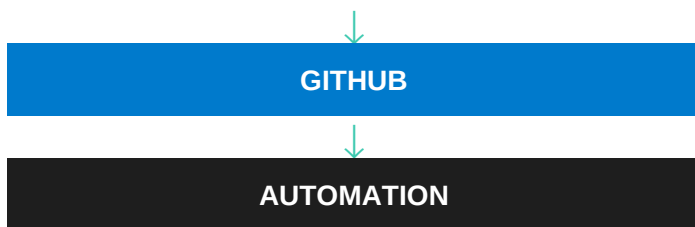
INSTALLING VS CODE

■ CONCEPT

Always download VS Code from its official site (code.visualstudio.com) — never a third-party mirror. Installers and version-specific steps change over time, so follow the official site's current instructions for your operating system.

THE VS CODE WORKFLOW





You write code in VS Code, run it using its built-in terminal, track changes with Git, back it up and share it on GitHub, and connect it into your wider automation systems (n8n, GitHub Actions, and beyond).

CORE FEATURES

- **Opening folders** — File → Open Folder loads your whole project, not just one file.
- **Creating files** — right-click in the file explorer panel → New File.
- **Extensions** — small add-ons that add support for languages and tools.
- **Integrated terminal** — View → Terminal opens a full terminal INSIDE VS Code, in your project's folder.
- **Python extension** — adds Python syntax highlighting, running, and debugging.
- **Git integration** — a built-in panel to stage, commit, and push changes.
- **Debugger** — lets you pause code mid-run and inspect variable values.
- **Source control** — the sidebar panel showing Git changes visually.
- **Workspace** — the folder(s) and settings for your current project.

- **Settings** — customise fonts, themes, formatting rules, and more.

RUNNING AND DEBUGGING SCRIPTS

With the Python extension installed and a .py file open, you can run it two ways:

- Click the  Run button in the top-right corner.
- Or open the integrated terminal and type: `python filename.py`

```
# hello.py
print("Hello Automation")
```

Expected output in the terminal panel: Hello Automation

■ DEBUG

If Run does nothing, confirm the Python extension is installed AND that VS Code has correctly detected a Python interpreter (bottom-right corner of the window should show a Python version, not a warning).

USEFUL EXTENSIONS FOR THIS BOOK

- **Python** (by Microsoft) — essential for Part 3 onward.
- **GitLens** — richer Git history and blame information.
- **Docker** — manage containers directly from VS Code (Part 13).
- **YAML** — syntax highlighting for GitHub Actions workflow files (Chapter 13).
- **REST Client** or similar — test API requests without leaving the editor.

■ QUICK TIP

You don't need to install every extension now. Add each one exactly when its chapter asks for it — a lighter setup stays faster and less confusing.

■ REMEMBER

VS Code is the one tool that stays open through this entire book — Python scripts, GitHub Actions YAML files, and even n8n workflow exports all get edited here.

EXERCISES

Quick Quiz

- What menu option opens an entire project folder in VS Code?
- How do you open the integrated terminal?
- What does the Python extension add to VS Code?
- Where does VS Code show your current Python interpreter?
- Name one VS Code extension useful for GitHub Actions.

Practical Exercise

Install VS Code, open your 'automation_practice' folder from Chapter 4, create a file called `hello.py` containing `print("Hello Automation")`, and run it using the integrated terminal.

Challenge

Install the Python extension, then use VS Code's Run button (not the terminal) to run `hello.py`, and confirm the output appears in the built-in terminal panel.

Interview Questions

- Why is an integrated terminal useful compared to a separate terminal window?

- What is the role of extensions in VS Code?
- How would you debug a Python script that isn't producing the expected output?

NEXT STEP: Volume 2 begins Part 3 — Python for Automation, where you'll write your first real automation scripts.

PART 3

PROGRAMMING FOR AUTOMATION

Python Fundamentals → Python For Automation
Just enough code to build, glue, and extend real workflows.

VS Code | VS Sir

CHAPTER 6

Python Fundamentals

■ CONCEPT

Every topic below follows the same shape: Concept → Syntax → Simple Example → Automation Example → Exercise.

VARIABLES & DATA TYPES

A **variable** stores a value under a name so you can reuse it.

```
file_name = "invoice.pdf"    # string
count = 12                   # integer (number)
price = 49.99                # float (decimal)
is_paid = False              # boolean (True/False)
```

Automation example: storing a folder path once, then reusing that variable in every file operation below it.

STRINGS & NUMBERS

```
name = "invoice"
full_name = name + ".pdf"    # "invoice.pdf"
message = f"File: {full_name}" # f-string: insert a variable into text
```

Automation example: f-strings build dynamic filenames, email subjects, and log messages constantly throughout this book.

LISTS, TUPLES, DICTIONARIES, SETS

- **List** — an ordered, changeable collection: `files = ['a.pdf', 'b.pdf']`

- **Tuple** — an ordered, unchangeable collection: `point = (10, 20)`
- **Dictionary** — key-value pairs: `student = {"name": "Asha", "marks": 92}`
- **Set** — unique, unordered values: `seen_ids = {101, 102, 103}`

Automation example: a dictionary is exactly how JSON data looks once loaded into Python — Chapter 8 builds directly on this.

CONDITIONS & LOOPS

```
for file in files:
    if file.endswith(".pdf"):
        print(f"Found PDF: {file}")
    else:
        print(f"Skipping: {file}")
```

Automation example: this exact pattern — loop through items, check a condition, act — is the core of almost every automation script in Chapter 7.

FUNCTIONS & MODULES

```
def rename_file(old_name, new_name):
    print(f"Renaming {old_name} to {new_name}")

import os    # a built-in module (reusable code li
brary)
```

A **module** is a file of reusable Python code you can import, either built into Python (like `os`) or installed separately (like `requests`).

EXCEPTIONS (ERROR HANDLING)

```
try:
    result = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

■ WARNING

Automation scripts run unattended — without `try/except`, one bad input can crash the entire process at 3am with no one watching.

FILE HANDLING, JSON & CSV (PREVIEW)

```
with open("log.txt", "a") as f:  
    f.write("Task completed\n")
```

Full JSON and CSV handling is covered in Chapter 7 and Chapter 8 — this is just the file-opening pattern they all build on.

ENVIRONMENT VARIABLES IN PYTHON

```
import os  
api_key = os.environ.get("API_KEY")
```

■ SECURITY

Never write a real API key directly in your script. Read it from an environment variable instead — this is covered fully in [security section](#)

■ REMEMBER

You don't need to memorise Python — you need to recognise these eight patterns (variables, strings, collections, conditions, loops, functions, exceptions, file handling) on sight.

EXERCISES

Quick Quiz

- What's the difference between a list and a tuple?
- What does a dictionary look like, and what real-world data format does it resemble?
- Why is try/except especially important for unattended automation?
- What does the f in an f-string do?
- Where should a real API key be stored, never in script code?

Practical Exercise

Write a script that loops through a Python list of five filenames and prints only the ones ending in '.csv'.

Challenge

Write a function that takes a dictionary representing a student (name, marks) and returns "Pass" if marks ≥ 40 , else "Fail", wrapped in a try/except in case 'marks' is missing.

Interview Questions

- Why does automation code need more error handling than a simple one-off script?
- What Python data structure maps most directly onto JSON?
- Explain the difference between a built-in and an installed module.

NEXT STEP: Chapter 7 applies these exact building blocks to real automation tasks — files, APIs, and scheduling.

CHAPTER 7

Python For Automation

READING, WRITING & RENAMING FILES

```
import os

for filename in os.listdir("downloads"):
    if filename.endswith(".jpg"):
        new_name = filename.replace(" ", "_")
        os.rename(f"downloads/{filename}", f"down
loads/{new_name}")
```

What it does: lists every file in 'downloads', and renames any .jpg file, replacing spaces with underscores.

FOLDER AUTOMATION

```
import shutil, os

os.makedirs("downloads/images", exist_ok=True)
for filename in os.listdir("downloads"):
    if filename.lower().endswith((".jpg", ".png")
):
        shutil.move(f"downloads/{filename}", f"do
wnloads/images/{filename}")
```

■ PRO PROJECT

This is Project 1 from Part 18: the Automatic File Organizer. Extend it with more folders (pdf/, docs/, videos/) once it works.

CSV PROCESSING

```
import csv

with open("students.csv", newline="") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["marks"])
```

DictReader turns each CSV row into a dictionary, using the header row as keys — directly reusing Chapter 6's dictionary pattern.

JSON PROCESSING (PREVIEW)

```
import json

with open("config.json") as f:
    config = json.load(f)
    print(config["api_url"])
```

Chapter 8 explains JSON's structure in full — this is simply how Python reads it.

HTTP REQUESTS & API CALLS

```
import requests    # pip install requests

response = requests.get("https://api.example.com/
weather")
data = response.json()
print(data["temperature"])
```

What it does: sends a request to a web address and reads back its JSON response. Chapters 9-10 explain exactly what's happening here.

EMAIL AUTOMATION (CONCEPT)

```
import smtplib
from email.mime.text import MIMEText

msg = MIMEText("Automated report attached.")
msg["Subject"] = "Daily Report"
msg["From"] = "YOUR_EMAIL"
msg["To"] = "recipient@example.com"
# Sending requires SMTP server credentials -- see
security sect for secure setup.
```

■ SECURITY

Email credentials must come from environment variables, never typed directly into the script — see [security section](#)

SCHEDULING SCRIPTS

```
import schedule, time

def daily_task():
    print("Running daily task...")

schedule.every().day.at("09:00").do(daily_task)
while True:
    schedule.run_pending()
    time.sleep(30)
```

■ QUICK TIP

For simple, reliable scheduling without keeping a script running 24/7, GitHub Actions' scheduled workflows (Chapter 13) or your operating system's own task scheduler are often better choices.

LOGGING

```
import logging
logging.basicConfig(filename="automation.log", level=logging.INFO)
logging.info("Task started")
logging.error("Task failed: file not found")
```

■ REMEMBER

An automation without logs is unauditable. If something breaks overnight, logs are the only record of what actually happened.

EXERCISES

Quick Quiz

- Which built-in module lists files in a folder?
- What does `csv.DictReader` turn each row into?

- Why shouldn't email credentials be typed directly into a script?
- What's an alternative to keeping a Python script running 24/7 for scheduling?
- Why does an unattended automation need logging?

Practical Exercise

Write a script that reads a CSV of student names and marks, and prints only the students who scored above 80.

Challenge

Extend the File Organizer example to sort files into pdf/, images/ and docs/ subfolders based on their extension, logging each move to a file.

Interview Questions

- Walk through what happens, step by step, when requests.get() is called.
- Why is CSV still widely used for automation despite JSON's flexibility?
- How would you make a file-renaming script safe to re-run multiple times?

NEXT STEP: Part 4 explains JSON, HTTP and APIs in full depth — the exact data and communication formats these scripts rely on.

PART 4

JSON → HTTP & Internet → APIs

How data actually travels between applications.

VS Code | VS Sir

CHAPTER 8

JSON

JSON

Simple meaning: JavaScript Object Notation — a lightweight text format for structuring data, using curly braces, keys and values.

Why it matters: It's the universal language almost every API, webhook, and n8n node uses to exchange data.

Example: `{"name": "Asha", "marks": 92}`

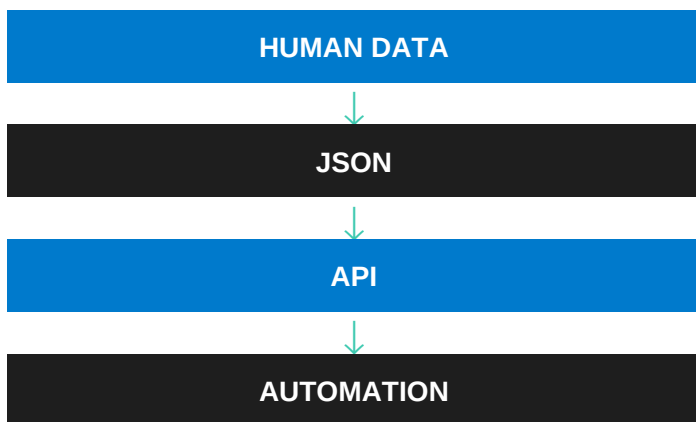
THE BUILDING BLOCKS

- **Object** — a set of key-value pairs inside `{ }`.
- **Key** — the label/name of a piece of data (always in quotes).
- **Value** — the actual data — text, a number, true/false, another object, or a list.
- **Array** — an ordered list of values inside `[ ]`.
- **Nested object** — an object stored as the value of another object's key.
- **Nested array** — an array stored as a value, often containing more objects.

```
{
  "student": "Asha",
  "marks": 92,
  "passed": true,
  "subjects": ["Physics", "Maths"],
  "address": {
    "city": "Delhi",
    "pincode": "110001"
  }
}
```

Here, 'subjects' is a nested array, and 'address' is a nested object — JSON can nest as deeply as needed to represent real-world data.

THE JSON JOURNEY



A form submission becomes structured JSON, which travels through an API, and finally powers an automated action — this is the data path behind almost every workflow you'll build from Volume 4 onward.

READING JSON IN PYTHON

```
import json

data = '{"name": "Asha", "marks": 92}'
student = json.loads(data)      # text -> Python
dictionary
print(student["name"])         # Asha

back_to_text = json.dumps(student)  # dictionary
-> text
```

- **json.loads()** — parses a JSON string into a Python dictionary/list.
- **json.dumps()** — converts a Python dictionary/list back into a JSON string.
- **json.load() / json.dump()** — the same, but reading/writing directly from a file.

■ DEBUG

A `JSONDecodeError` almost always means the JSON text is malformed — check for a missing comma, an unclosed bracket, or single quotes used instead of double quotes (JSON requires double quotes).

■ REMEMBER

If you can read a Python dictionary, you can already read JSON — they're structured almost identically.

EXERCISES

Quick Quiz

- What are the four building blocks of JSON?

- What Python function turns a JSON string into a dictionary?
- Why must JSON keys and string values use double quotes?
- What's a 'nested object'?
- Name one real system that sends data as JSON.

Practical Exercise

Write a JSON object describing a book (title, author, price, in_stock true/false) and load it into Python with `json.loads()`.

Challenge

Extend the book JSON to include a nested array of 'reviews', each review being an object with 'user' and 'rating', then write Python code to print every reviewer's name.

Interview Questions

- Why is JSON preferred over CSV for nested or hierarchical data?
- What's the difference between `json.loads()` and `json.load()`?
- How would you debug a `JSONDecodeError` in a script processing API responses?

NEXT STEP: Chapter 9 explains HTTP — how this JSON data actually travels across the internet.

CHAPTER 9

HTTP & Internet

Client

Simple meaning: The program that starts a request — your browser, phone app, or automation script.

Why it matters: Every automation you build acts as a client when it calls an API.

Example: Your n8n workflow requesting today's weather.

Server

Simple meaning: The program that receives a request and sends back a response.

Why it matters: Understanding client/server is the foundation of understanding any API call.

Example: A weather service's server, replying with today's temperature.

HTTP

Simple meaning: HyperText Transfer Protocol — the standard set of rules for clients and servers to communicate.

Why it matters: Virtually every API and webhook in this book runs over HTTP.

Example: `http://example.com`

HTTPS

Simple meaning: HTTP, but encrypted for security.

Why it matters: Automation should always use HTTPS, especially when sending API keys or personal data.

Example: `https://api.example.com`

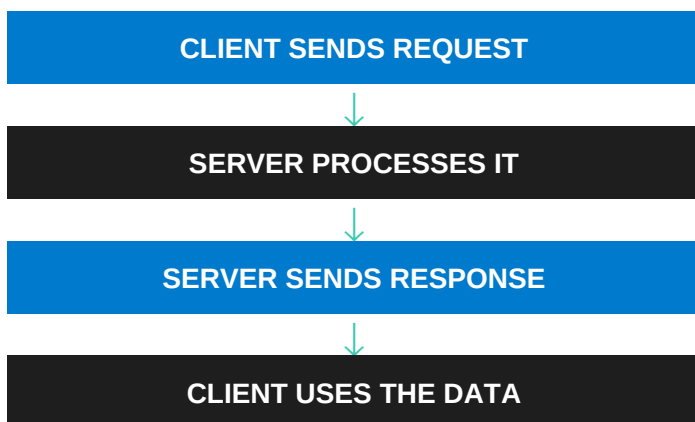
URL

Simple meaning: Uniform Resource Locator — the address of a specific resource on the internet.

Why it matters: It's what you point your automation at.

Example: `https://api.example.com/students/42`

REQUEST & RESPONSE



- **Headers** — metadata about the request/response (e.g. content type, authentication).
- **Body** — the actual data being sent, often JSON.
- **Query parameters** — extra options appended to a URL after a `?`.

```
https://api.example.com/students?class=10&sort;=marks
#                               ^ path               ^ query
y parameters
```

HTTP METHODS

- **GET** — retrieve data, without changing anything.
- **POST** — create something new (e.g. submit a form).
- **PUT / PATCH** — update something existing (PUT replaces it fully, PATCH updates part of it).
- **DELETE** — remove something.

```
GET    /students/42          # fetch student 42
POST   /students           # create a new student
PUT    /students/42        # replace student 42's
      full record
DELETE /students/42        # delete student 42
```

STATUS CODES

- **200 OK** — the request succeeded.
- **201 Created** — something new was successfully created.
- **400 Bad Request** — the request was malformed.
- **401 Unauthorized** — missing or invalid authentication.
- **404 Not Found** — the requested resource doesn't exist.
- **429 Too Many Requests** — you've hit a rate limit.
- **500 Internal Server Error** — something broke on the server's side.

■ DEBUG

Always check the status code first when an automation fails on an API call — it immediately narrows down whether the problem is your request (4xx) or the server (5xx).

■ REMEMBER

GET reads, POST creates, PUT/PATCH updates, DELETE removes. Almost every API you'll ever touch follows this same pattern.

EXERCISES

Quick Quiz

- What's the difference between a client and a server?
- Which HTTP method would you use to update part of an existing record?
- What does a 404 status code mean?
- What does a 401 status code usually indicate is wrong?
- Why should automation always use HTTPS instead of HTTP?

Practical Exercise

Using Python's requests library, send a GET request to any public test API (e.g. <https://api.github.com>) and print the status code.

Challenge

Write a function that takes a status code and returns a plain-English explanation of what it means, covering at least 5 codes from this chapter.

Interview Questions

- Explain the request/response cycle in your own words.
- What's the difference between PUT and PATCH?
- How would you diagnose a 500 error versus a 400 error?

NEXT STEP: Chapter 10 puts HTTP and JSON together into full API workflows.

CHAPTER 10

APIs

API

Simple meaning: A communication bridge that allows two software systems to exchange information.

Why it matters: APIs are how virtually every automation tool in this book — n8n, GitHub, AI models — talks to the outside world.

Example: Application A → API → Application B

REST API

Simple meaning: The most common API style, built directly on HTTP methods (GET/POST/PUT/DELETE) and URLs.

Why it matters: Almost every service you'll connect n8n to (Chapters 14-20) exposes a REST API.

Example: GET <https://api.example.com/students>

AUTHENTICATION

- **API key** — a single secret string identifying your app to the API.
- **OAuth** — a secure standard for a user to grant your app limited access to their account, without sharing their password.
- **Token** — a temporary credential (often issued after OAuth login) proving you're authorised.

```
headers = {"Authorization": "Bearer YOUR_API_KEY"}
response = requests.get(url, headers=headers)
```

■ SECURITY

Never hard-code `YOUR_API_KEY` directly. Always load it from an environment variable or your automation tool's secure credential store.

RATE LIMITS & PAGINATION

Rate limit

Simple meaning: The maximum number of requests an API allows in a given time period.

Why it matters: Exceeding it returns a 429 error and can temporarily block your app.

Example: “100 requests per minute.”

Pagination

Simple meaning: Splitting a large result set across multiple pages/requests.

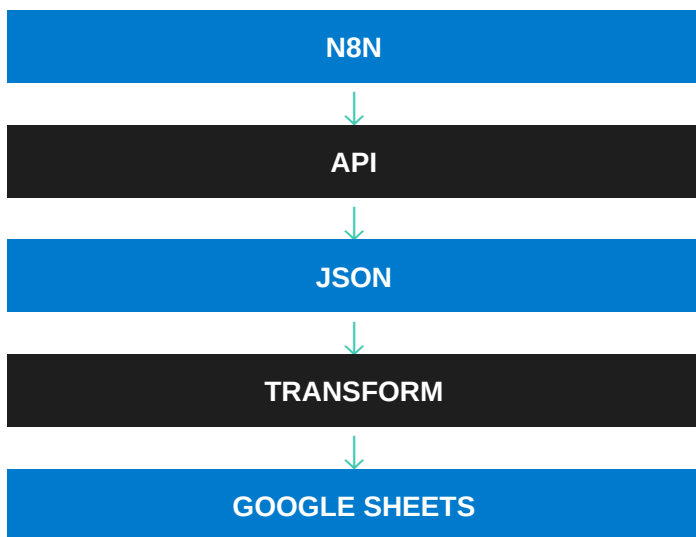
Why it matters: APIs rarely return everything at once — your automation must loop through pages to get all the data.

Example: `?page=1&per_page=50`, then
`?page=2&per_page=50`

ERROR HANDLING FOR APIs

```
response = requests.get(url, headers=headers)
if response.status_code == 200:
    data = response.json()
elif response.status_code == 429:
    print("Rate limited -- wait and retry")
else:
    print(f"API error: {response.status_code}")
```

A COMPLETE API WORKFLOW



This exact shape — call an API, receive JSON, reshape it, send it somewhere useful — is the backbone of Part 6's entire n8n course.

■ PRO PROJECT

Preview of Project 6 (Part 18): build an API Data Collector that calls a public API on a schedule, and appends the results to a CSV file, handling rate limits gracefully.

■ REMEMBER

An API is just a client/server conversation (Chapter 9), carrying JSON (Chapter 8), protected by authentication. Once you see that, every new API you encounter looks familiar.

EXERCISES

Quick Quiz

- What's the difference between an API key and an OAuth token?
- What HTTP status code signals you've hit a rate limit?
- Why do APIs use pagination instead of returning everything at once?
- Where should an API key be stored in a real project?
- What four things combine to make a 'complete API workflow' in this chapter?

Practical Exercise

Call a public API of your choice with Python's requests library, check the status code, and print a friendly message for both success and failure cases.

Challenge

Write a script that loops through paginated results from a public API (check its docs for a 'page' or 'cursor' parameter) and combines all pages into one Python list.

Interview Questions

- Explain REST API authentication options and when you'd use each.
- How would you design a script to gracefully handle a rate-limited API?

- Walk through the full journey of data from an API call to a Google Sheet.

NEXT STEP: Volume 3 covers Git, GitHub, and GitHub Actions — version control and automated pipelines for everything you build.

PART 5

GITHUB

Git Fundamentals → GitHub → GitHub Actions
Version control, collaboration, and free automation runners.

VS Code | VS Sir

CHAPTER 11

Git Fundamentals

Git

Simple meaning: A version control system that tracks every change made to your files over time, running entirely on your own computer.

Why it matters: It lets you undo mistakes, see exactly what changed and when, and work safely without fear of losing work.

Example: Every professional codebase, and most n8n workflow backups, use Git.

GitHub

Simple meaning: A cloud website that hosts Git repositories, adding collaboration features (issues, pull requests, Actions) on top of plain Git.

Why it matters: It's where you back up, share, and automate around your code.

Example: github.com

■ QUICK TIP

Git is the tool; GitHub is one popular place to host what Git tracks. You can use Git without ever touching GitHub, but GitHub needs Git underneath it.

CORE CONCEPTS

Repository (repo)

Simple meaning: A project folder that Git is tracking the history of.

Why it matters: Everything in this book — scripts, n8n exports, YAML files — lives inside a repository.

Example: my-automation-project/

Commit

Simple meaning: A saved snapshot of your project at a specific point in time, with a message describing what changed.

Why it matters: Commits are how you build an undoable, readable history of your work.

Example: "Add file organizer script"

Branch

Simple meaning: A separate, parallel line of work, based on your project at a point in time.

Why it matters: Branches let you experiment or build a new feature without risking your main, working version.

Example: main, feature/email-automation

Merge

Simple meaning: Combining changes from one branch back into another.

Why it matters: Once a new feature works, you merge it back into your main branch.

Example: Merging feature/email-automation into main.

Clone

Simple meaning: Downloading a full copy of a remote repository (with its whole history) onto your own computer.

Why it matters: It's how you get someone else's project — or your own GitHub repo — onto a new machine.

Example: `git clone https://github.com/user/repo.git`

Pull

Simple meaning: Downloading the latest changes from a remote repository into your local copy.

Why it matters: Keeps your local files in sync with what's on GitHub.

Example: `git pull`

Push

Simple meaning: Uploading your local commits to a remote repository.

Why it matters: This is how your local work actually reaches GitHub.

Example: `git push`

Pull Request (PR)

Simple meaning: A request to merge your branch's changes into another branch, usually reviewed before merging.

Why it matters: PRs are the standard way teams review and discuss code changes safely.

Example: "Merge feature/email-automation into main"

.gitignore

Simple meaning: A file listing which files/folders Git should NEVER track.

Why it matters: It's essential for keeping secrets (.env files) and clutter (temporary files) out of your repository.

Example: `.env __pycache__/*log`

README

Simple meaning: A file (usually README.md) describing what a repository does and how to use it.

Why it matters: It's the first thing anyone sees when they open your repository.

Example: README.md

THE GIT WORKFLOW, VISUALLY

EDIT FILES (working directory)



git add (staging area)



git commit (local repository)



git push (remote / GitHub)

```
git add script.py           # stage the file
git commit -m "Add script"  # save a snapshot
git push                    # upload to GitHub
```

■ SECURITY

.gitignore is your first line of defense against leaking secrets. Add .env to it BEFORE your first commit — once a secret is pushed to GitHub, treat it as compromised, even if you delete it later.

■ REMEMBER

Commit often, with clear messages. A messy history is confusing; a clean one is a map of exactly how your project was built.

EXERCISES

Quick Quiz

- What's the difference between Git and GitHub?
- What is a commit, and why does the message matter?
- What's the purpose of a branch?
- What does .gitignore do, and why is it important for security?
- What's the difference between git pull and git push?

Practical Exercise

Install Git, initialise a new repository in your automation_practice folder from Volume 1 (git init), and make your first commit.

Challenge

*Create a .gitignore file that excludes .env, __pycache__/ and *.log, then confirm (with git status) that a test .env file is correctly ignored.*

Interview Questions

- Why is version control essential even for solo automation projects?
- Explain the staging area's purpose in the Git workflow.
- What would you do if you accidentally committed a real API key?

NEXT STEP: Chapter 12 moves this workflow onto GitHub itself.

CHAPTER 12

GitHub

GETTING STARTED

- **Creating an account** — sign up at github.com with an email address.
- **Creating a repository** — click 'New', name it, choose Public or Private.
- **Uploading a project** — either drag-and-drop files in the browser, or push from Git (Chapter 11).

■ CONCEPT

A Public repository is visible to anyone; a Private repository is visible only to you and people you invite. Never store real secrets in either — privacy settings don't replace proper credential management (Part 15).

CLONING & CONNECTING VS CODE

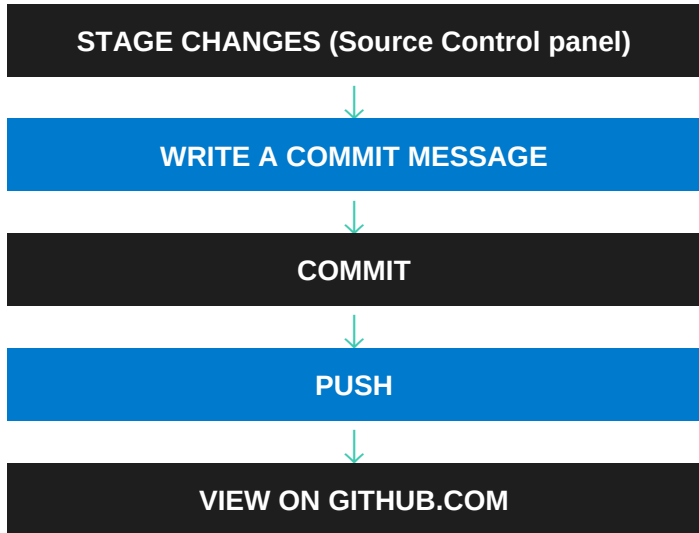
```
git clone https://github.com/your-username/your-r  
epo.git
```

Open the cloned folder in VS Code (File → Open Folder), and use the Source Control panel (Chapter 5) to commit and push directly from the editor, without leaving it.

THE COMMIT / PUSH WORKFLOW IN PRACTICE

EDIT IN VS CODE





BRANCHING & PULL REQUESTS

```
git checkout -b feature/new-script # create and  
switch to a new branch  
git push -u origin feature/new-script
```

Once pushed, GitHub's website will offer to open a Pull Request, comparing your branch to main — the standard way to review changes before merging.

ISSUES

Issues are GitHub's built-in task/bug tracker — useful even for solo projects, as a running list of automation ideas, bugs, and improvements you want to make.

RELEASES

A **release** marks a specific, stable snapshot of your repository (e.g. "v1.0") that others can reliably download, separate from your constantly-changing main branch.

A REAL AUTOMATION WORKFLOW

WRITE SCRIPT LOCALLY (VS Code)



COMMIT & PUSH TO GITHUB



OPEN AN ISSUE if something's broken



FIX ON A BRANCH



OPEN A PULL REQUEST



MERGE



TAG A RELEASE when stable

■ QUICK TIP

Even working completely alone, using branches and pull requests for anything non-trivial gives you a reviewable history and an easy way to back out of a bad change.

■ REMEMBER

GitHub is not just backup — it's also the trigger source for GitHub Actions, covered next, turning every push into a potential automation event.

EXERCISES

Quick Quiz

- What's the difference between a Public and Private repository?
- How do you connect VS Code to a GitHub repository?
- What is a Pull Request used for?
- What's the purpose of GitHub Issues?
- What does tagging a Release accomplish that a normal commit doesn't?

Practical Exercise

Create a new GitHub repository, clone it into VS Code, add a README.md describing your automation practice project, and push it.

Challenge

Create a new branch, make a small change, push the branch, and open a Pull Request comparing it to main on GitHub.com.

Interview Questions

- Why do teams use Pull Requests instead of pushing directly to main?

- What's the practical difference between an Issue and a Pull Request?
- How would you structure a repository for a growing automation project?

NEXT STEP: Chapter 13 turns your GitHub repository into an automation engine of its own, with GitHub Actions.

CHAPTER 13

GitHub Actions

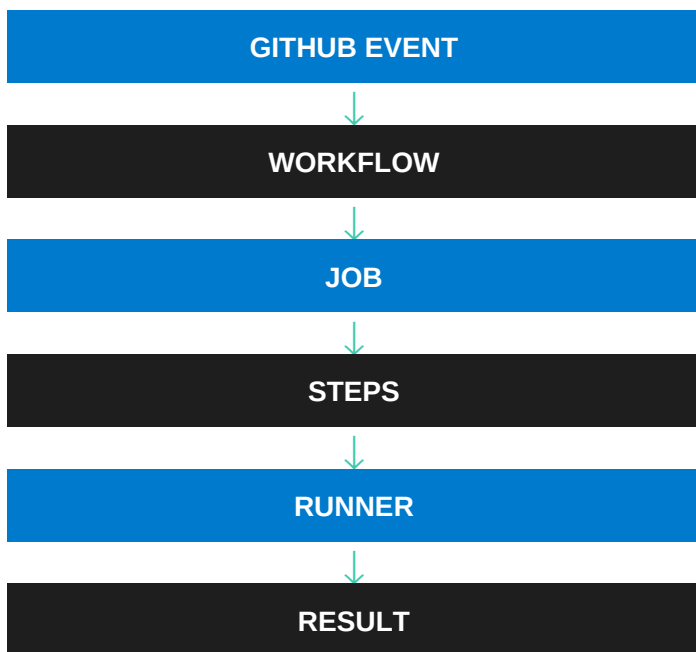
GitHub Actions

Simple meaning: GitHub's built-in automation system: it runs code automatically in response to events in your repository.

Why it matters: It's a free, reliable way to automate testing, deployment, and scheduled scripts, without hosting your own server.

Example: Automatically run a Python script every time you push new code.

THE CORE PIPELINE



- **Workflow** — the whole automation, defined in a YAML file.
- **Trigger / Event** — what starts the workflow (a push, a schedule, a manual click).
- **Job** — a group of steps that run together on one runner.
- **Step** — one individual action or command within a job.
- **Runner** — the actual (usually temporary, cloud-hosted) machine that executes your steps.
- **Action** — a reusable packaged step someone else has built (e.g. 'checkout code').

YAML

Simple meaning: A human-readable text format using indentation, used to configure GitHub Actions workflows (and much more).

Why it matters: Every GitHub Actions workflow file is written in YAML — indentation matters, unlike in Python's braces-optional style, but similar to Python's own indentation rules.

Example: `.github/workflows/automation.yml`

FILE LOCATION

```
.github/  
  workflows/  
    automation.yml
```

GitHub automatically detects any YAML file placed inside **.github/workflows/** in your repository — no other setup step is required.

A FIRST WORKFLOW, EXPLAINED LINE BY LINE

```
name: Run Automation Script

on:
  push:
    branches: [main]
  schedule:
    - cron: "0 9 * * *"      # 9 AM UTC daily
  workflow_dispatch:        # allows manual run button

jobs:
  run-script:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: python automation_script.py
```

- **name:** the workflow's display name on GitHub.
- **on:** the triggers — here, every push to main, a daily 9 AM schedule, and a manual button.
- **jobs / run-script:** one job, named 'run-script'.
- **runs-on:** the runner OS — a fresh Ubuntu Linux virtual machine each time.
- **steps:** checkout the code, set up Python, install dependencies, then run the script.

■ CONCEPT

Always check GitHub's current official documentation for the latest supported action versions (e.g. actions/checkout@v4) — these version numbers update over time.

SECRETS & ENVIRONMENT VARIABLES

```
- run: python send_report.py
  env:
    API_KEY: ${ secrets.API_KEY }
```

■ SECURITY

Add secrets under your repository's Settings → Secrets and variables → Actions. Never write a real key directly in the YAML file — it would be visible to anyone who can read your repository.

ARTIFACTS

Artifacts are files a workflow produces (e.g. a generated report) that you can download afterward, using the actions/upload-artifact action.

CI AND CD

CI (Continuous Integration)

Simple meaning: Automatically testing code every time it changes.

Why it matters: It catches bugs immediately, before they reach production.

Example: A workflow that runs your test suite on every push.

CD (Continuous Deployment)

Simple meaning: Automatically deploying code once it passes its tests.

Why it matters: It removes manual, error-prone deployment steps.

Example: A workflow that deploys your app automatically after tests pass on main.

■ DEBUG

If a workflow doesn't trigger, double-check: the YAML file is inside `.github/workflows/`, the file extension is `.yml` or `.yaml`, and the indentation is consistent (YAML is strict about spacing).

■ REMEMBER

GitHub Actions turns 'push code' into a trigger, exactly like n8n turns 'form submitted' into a trigger — same automation thinking from Chapter 2, a different tool.

EXERCISES

Quick Quiz

- Where must a GitHub Actions workflow file be located?
- What are the three triggers used in this chapter's example workflow?
- What does 'runs-on: ubuntu-latest' mean?
- How should a real API key be made available to a workflow?
- What's the difference between CI and CD?

Practical Exercise

Create a workflow file that runs 'python --version' every time you push to main, and confirm it appears under your repository's Actions tab.

THANK YOU FOR READING

AUTOMATION MASTERY

Learn Automation from Zero • Python • APIs • Git • GitHub

KEEP LEARNING. KEEP BUILDING.

This is the beginning of your automation journey.
More practical volumes, projects and AI-powered learning resources
are planned for the next stages.

OUR FEEDBACK HELPS US BUILD BETTER BOOKS

Send your review, correction or suggestion on WhatsApp:

+91 96024 05311

Message: "Automation Mastery Review — Chapter / Page / Issue"

COMING NEXT

More practical automation + AI projects • workflows • real-world builds

Vijay Kumar Sharma (VS Sir)

AI-enabled educator • Physics • Technology • Practical Learning